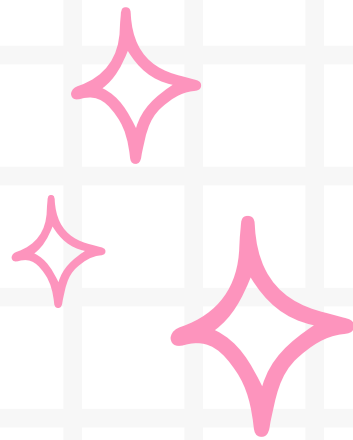
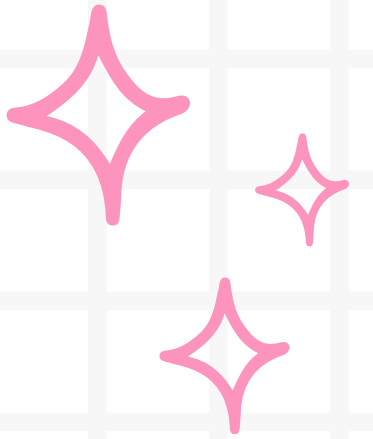


Marta González González

POCKET PROFESSOR

MODULE 20



INTRODUCTION

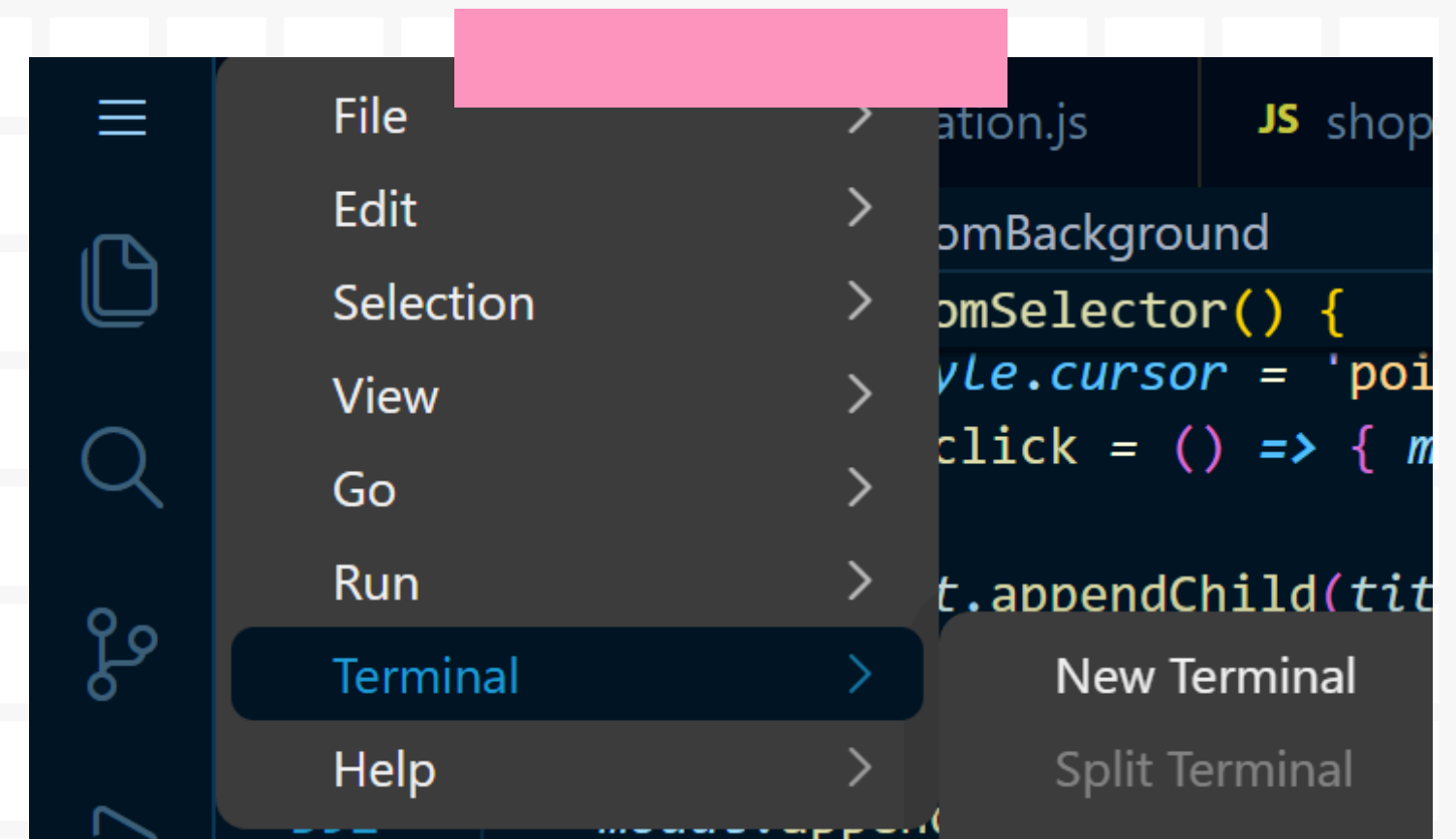
For this assignment, I created "Pocket Professor", a simulation web game where the user can interact with virtual professors, give them food, toys, or take them to different scenarios.

For the development, I used artificial intelligence as a support tool. It helped me solve logic doubts, correct syntax errors in JavaScript, and structure the game's base more efficiently. In this presentation, I will detail the process I followed in the Git command line to manage versions and the project's progress securely

PREPARATION AND iNiTiALiZATiON

Opening the terminal and initializing the repository (`git init`)

Before starting with version control, it was necessary to set up the workspace. To do this, I opened the integrated terminal in Visual Studio Code by going to the top menu, clicking on "Terminal", and selecting "New Terminal". This provided me with the command interface directly within the project editor.



PREPARATION AND INITIALIZATION

Opening the terminal and initializing the repository (`git init`)

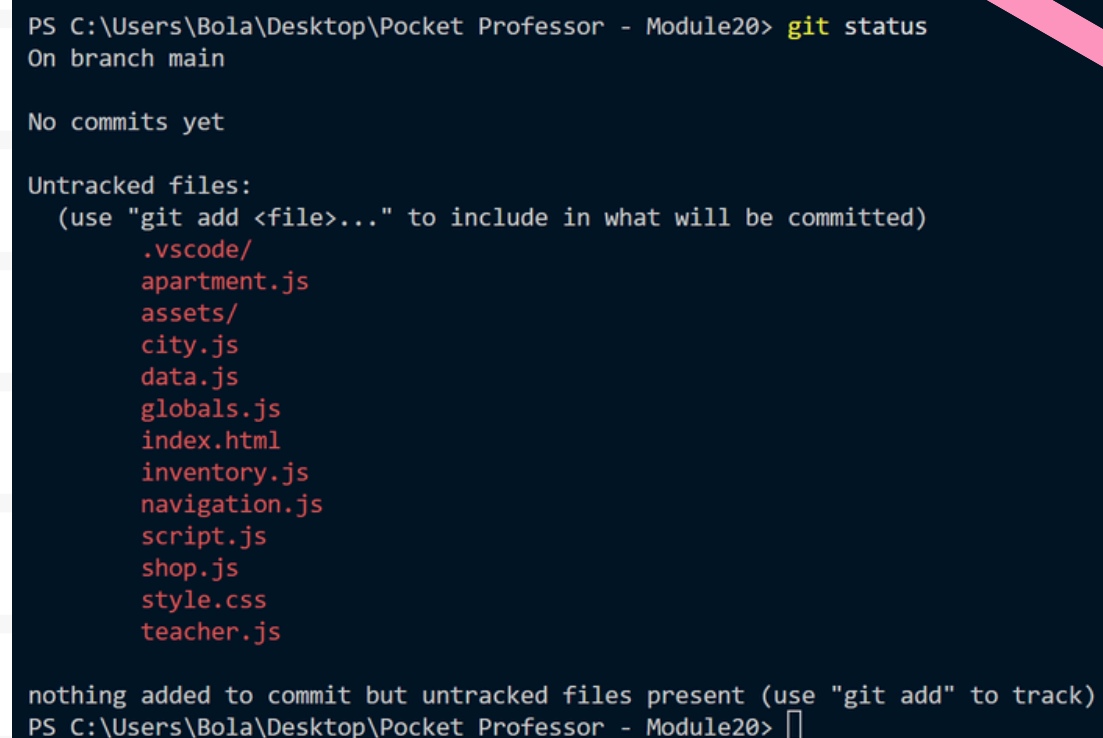
Once open, the first step was to execute the `git init` command. This command initializes the version control system in the current directory. When executed, it generates a hidden folder (`.git`) that stores and manages the entire history of the project. Without this initial step, it is not possible to track file changes

```
PS C:\Users\Bola\Desktop\Pocket Professor - Module20> git init
Initialized empty Git repository in C:/Users/Bola/Desktop/Pocket Professor - M
odule20/.git/
PS C:\Users\Bola\Desktop\Pocket Professor - Module20> 
```

THE FIRST OFFICIAL SAVE

With the first project files created, I proceeded to record the initial version in the history using three commands:

1. First I used `git status` to check which new or modified files Git had detected in the directory
2. Then I used `git add .` to add all those files to the staging area. This selects the files that will be part of the next save

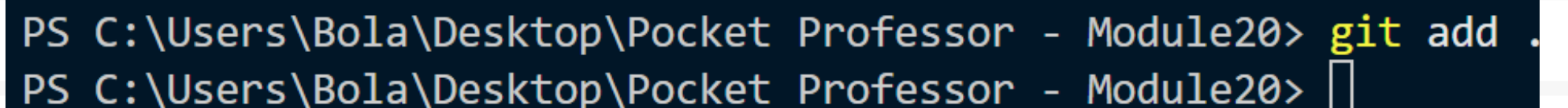
A terminal window showing the output of the 'git status' command. The output indicates that there are no commits yet and lists several untracked files in the directory. A pink arrow points to the 'git status' command in the first line of the terminal output.

```
PS C:\Users\Bola\Desktop\Pocket Professor - Module20> git status
On branch main

No commits yet

Untracked files:
  (use "git add <file>..." to include in what will be committed)
    .vscode/
    apartment.js
    assets/
    city.js
    data.js
    globals.js
    index.html
    inventory.js
    navigation.js
    script.js
    shop.js
    style.css
    teacher.js

nothing added to commit but untracked files present (use "git add" to track)
PS C:\Users\Bola\Desktop\Pocket Professor - Module20>
```

A terminal window showing the execution of the 'git add .' command. The command is entered and executed, with a prompt character visible at the end of the second line.

```
PS C:\Users\Bola\Desktop\Pocket Professor - Module20> git add .
PS C:\Users\Bola\Desktop\Pocket Professor - Module20>
```

THE FIRST OFFICIAL SAVE

With the first project files created, I proceeded to record the initial version in the history using three commands:

3. Finally, I executed `git commit -m "Commit inicial"`.

I used this command to definitively record the files prepared in the previous step. The `-m` parameter allows attaching a descriptive message ("Commit inicial") to quickly identify the changes of this version when reviewing the history in the future

```
PS C:\Users\Bola\Desktop\Pocket Professor - Module20> git commit -m "Commit inicial: Código de todo el funcionamiento de Pocket Proffesor"
[main (root-commit) d78dbff] Commit inicial: Código de todo el funcionamiento de Pocket Proffesor
61 files changed, 4046 insertions(+)
create mode 100644 .vscode/settings.json
create mode 100644 apartment.js
create mode 100644 assets/audio/cursor.mp3
create mode 100644 assets/audio/soundtrack.mp3
create mode 100644 assets/fonts/DreamLife-V1.0.0_by_MaxiGamer.otf
create mode 100644 assets/fonts/DreamLife-V1.0.0_by_MaxiGamer.ttf
create mode 100644 assets/images/agua.png
create mode 100644 assets/images/boli.png
create mode 100644 assets/images/cafe.png
create mode 100644 assets/images/carne.png
create mode 100644 assets/images/chuches.png
create mode 100644 assets/images/clase.jpg
create mode 100644 assets/images/cursor1.cur
```

The `commit` command permanently records the current state of the files, creating a safe restoration point in the project's history.

CREATING A TESTING ENVIRONMENT

Branches in Git (`git branch` & `git checkout`)

During development, I decided to add a new feature to the game. To avoid altering the main code that was already stable, I first created a new branch using the command `git branch feature-update`. Then, I switched to this new environment using `git checkout feature-update`

The `git branch` command creates a new branch (a parallel line of development) without modifying the original code. The `git checkout` command switches the working directory to that specific branch. Together, they create an isolated environment: any modification or error that occurs within this branch will not affect the main version of the project at all

```
PS C:\Users\Bola\Desktop\Pocket Professor - Module20> git branch feature-update
PS C:\Users\Bola\Desktop\Pocket Professor - Module20> 
```

```
PS C:\Users\Bola\Desktop\Pocket Professor - Module20> git checkout feature-update
Switched to branch 'feature-update'
PS C:\Users\Bola\Desktop\Pocket Professor - Module20> 
```

SAVING THE NEW CHANGES

Working on the new feature

Inside this new development branch, I programmed the necessary code changes to implement the planned improvement. Once I verified that the feature operated correctly, I repeated the version recording process. I added them to the staging area with `git add .`, and registered the changes in the local history with a new `git commit`, including an explanatory message of the update

```
PS C:\Users\Bola\Desktop\Pocket Professor - Module20> git add .  
PS C:\Users\Bola\Desktop\Pocket Professor - Module20> █
```

```
PS C:\Users\Bola\Desktop\Pocket Professor - Module20> git commit -m "Modificación del cursor cuando agarras a un personaje"  
[feature-update f00fc6c] Modificación del cursor cuando agarras a un personaje  
3 files changed, 14 insertions(+)  
PS C:\Users\Bola\Desktop\Pocket Professor - Module20> █
```

INTEGRATING THE CODE

Merging the code (`git merge`)

After verifying the stability of the new feature in the testing branch, the next step was to incorporate it into the project's official code. To achieve this, I first returned to the main branch by executing `git checkout main`. Next, I used the command `git merge feature-update`

The `merge` command is used to merge histories. Its function is to take all the work developed in the secondary branch and safely integrate it with the main code. This way, the official version is updated with the new functionality

```
PS C:\Users\Bola\Desktop\Pocket Professor - Module20> git checkout main
Switched to branch 'main'
PS C:\Users\Bola\Desktop\Pocket Professor - Module20> █
```

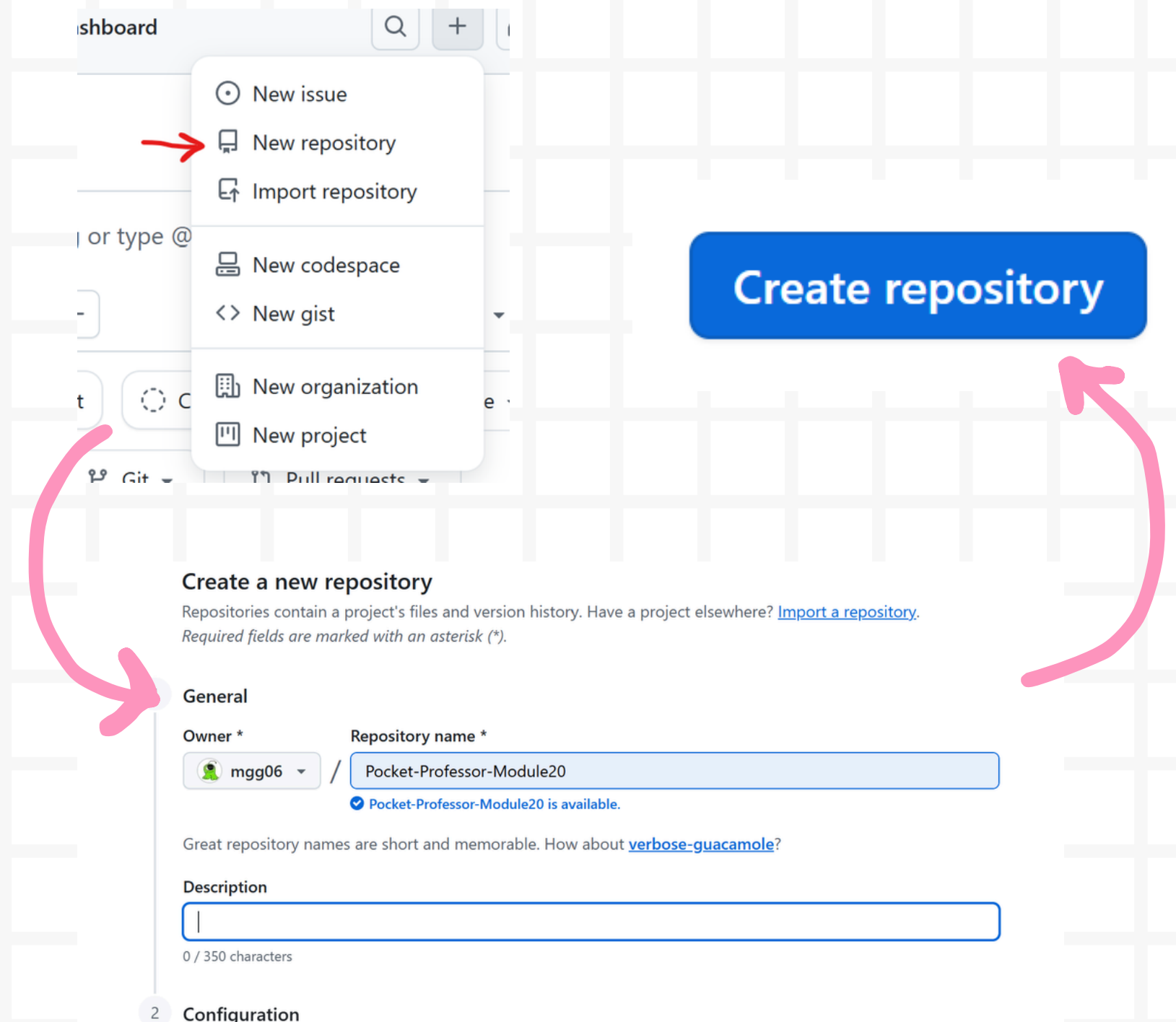
```
PS C:\Users\Bola\Desktop\Pocket Professor - Module20> git merge feature-update
Updating d78dbff..f00fc6c
Fast-forward
 navigation.js | 7 ++++++
 style.css     | 1 +
 teacher.js    | 6 +++++
 3 files changed, 14 insertions(+)
PS C:\Users\Bola\Desktop\Pocket Professor - Module20> █
```

CREATING THE CLOUD REPOSITORY

Preparing the environment on GitHub

To have an external backup and facilitate the project's distribution, I proceeded to create a repository on the **GitHub platform**.

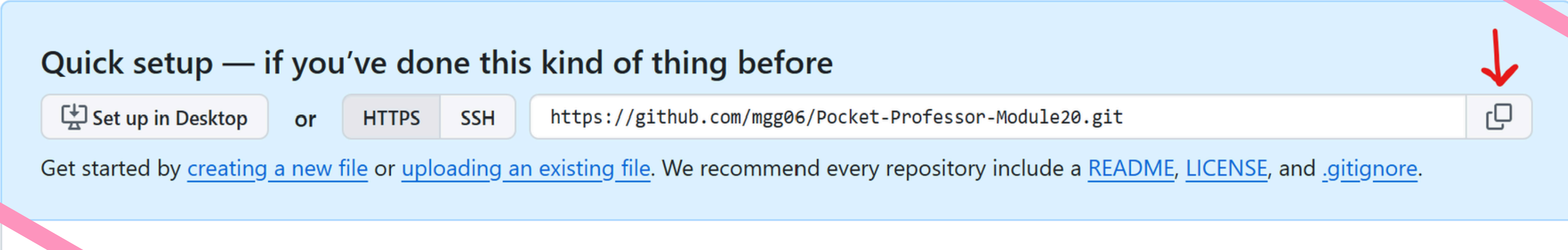
The process consisted of logging into my GitHub account, clicking the "+" icon located in the top right corner, and selecting the "New repository" option. Next, I entered the project name in the corresponding field and directly pressed the green "Create repository" button located at the bottom



CREATING THE CLOUD REPOSITORY

Preparing the environment on GitHub

After creation, the platform directed me to a new screen showing the connection instructions. From this page, I copied the web link (URL) associated with my new repository to link it with my local environment



Quick setup — if you've done this kind of thing before

 Set up in Desktop or HTTPS SSH 

Get started by [creating a new file](#) or [uploading an existing file](#). We recommend every repository include a [README](#), [LICENSE](#), and [.gitignore](#).

The screenshot shows a light blue box with a title 'Quick setup — if you've done this kind of thing before'. Below the title are two options: 'Set up in Desktop' with a desktop icon and 'or' followed by 'HTTPS' and 'SSH' buttons. To the right of these buttons is a text input field containing the URL 'https://github.com/mgg06/Pocket-Professor-Module20.git' and a copy icon. Below this is a paragraph of text: 'Get started by [creating a new file](#) or [uploading an existing file](#). We recommend every repository include a [README](#), [LICENSE](#), and [.gitignore](#).' A red arrow points to the copy icon, and pink diagonal lines are present in the corners of the box.

LOCAL AND REMOTE SYNCHRONIZATION

Branches in Git (`git branch` & `git checkout`)

Once the space on GitHub was created and the link copied, I returned to the Visual Studio Code terminal to link the local repository with the cloud server. To achieve this connection, I executed the `git remote add origin` command followed by the URL I had just copied. Subsequently, I used the `git push -u origin main` instruction to send the data

```
PS C:\Users\Bola\Desktop\Pocket Professor - Module20> git remote add origin https://github.com/mgg06/Pocket-Professor-Module20.git
PS C:\Users\Bola\Desktop\Pocket Professor - Module20> |
```

The `push` command is used to send and synchronize the files and the complete version history from the local working environment to the remote GitHub server, thus completing the project upload

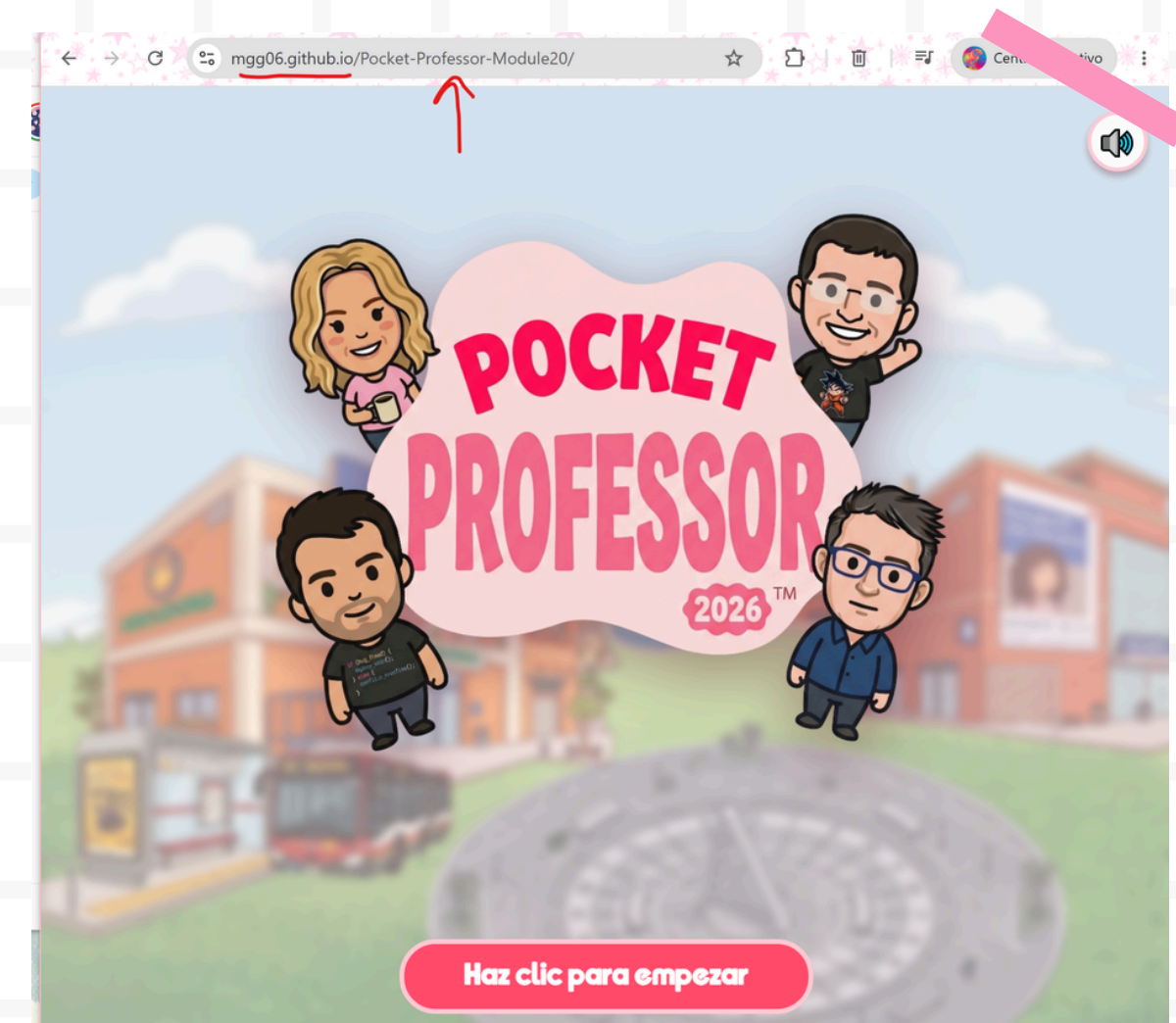
```
PS C:\Users\Bola\Desktop\Pocket Professor - Module20> git push -u origin main
Enumerating objects: 73, done.
Counting objects: 100% (73/73), done.
Delta compression using up to 16 threads
Compressing objects: 100% (71/71), done.
Writing objects: 100% (73/73), 32.85 MiB | 8.43 MiB/s, done.
Total 73 (delta 9), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (9/9), done.
To https://github.com/mgg06/Pocket-Professor-Module20.git
 * [new branch]      main -> main
branch 'main' set up to track 'origin/main'.
PS C:\Users\Bola\Desktop\Pocket Professor - Module20> |
```

PUBLISHING THE WEBSITE

Activating GitHub Pages

To meet the web accessibility requirement, I configured GitHub's free deployment environment. To reach this option, I navigated to the "Settings" tab within my GitHub repository. On the left side menu, I selected the "Pages" section. In the configuration section, I selected the "main" branch as the source of the code and saved the changes

GitHub Pages processes the repository files (HTML, CSS, and JS) and publishes them as a functional, publicly accessible web page. This allows any user to interact with the game through a browser without needing to download the source code



<https://mgg06.github.io/Pocket-Professor-Module20/>

THANK
YOU

